

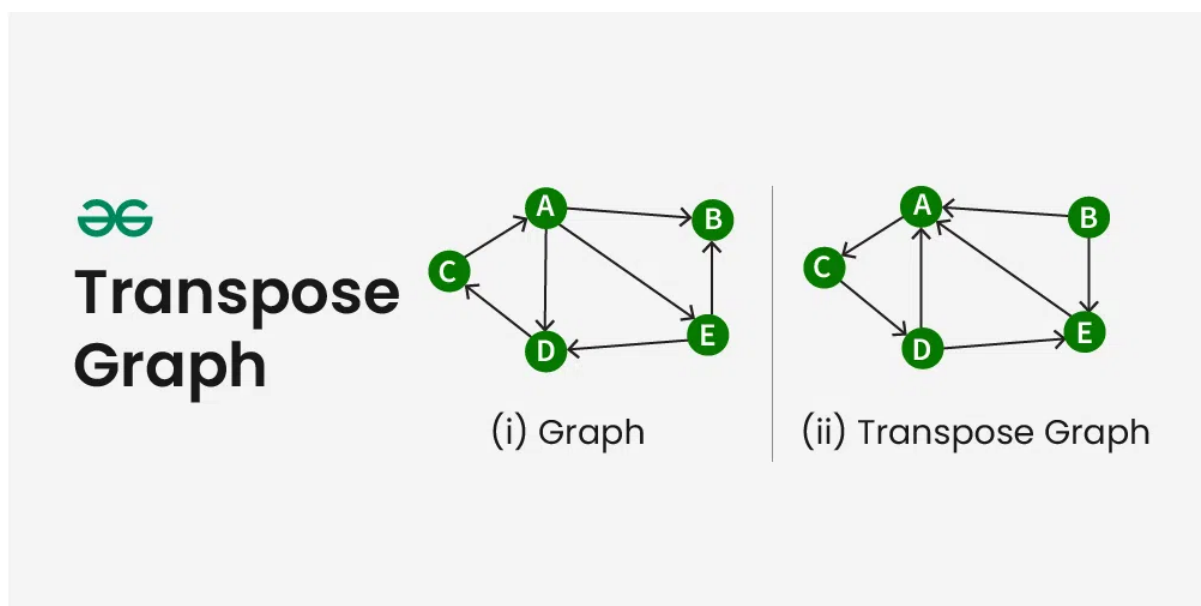
Transpose graph

Last Updated : 26 Feb, 2024



Transpose of a directed graph G is another directed graph on the same set of vertices with all of the edges reversed compared to the orientation of the corresponding edges in G . That is, if G contains an edge (u, v) then the converse/transpose/reverse of G contains an edge (v, u) and vice versa. Given a graph (represented as adjacency list), we need to find another graph which is the transpose of the given graph.

Example:



Input : figure (i) is the input graph.

Output : figure (ii) is the transpose graph of the given graph.

We traverse the adjacency list and as we find a vertex v in the adjacency list of vertex u which indicates an edge from u to v in main graph, we just add an edge from v to u in the transpose graph i.e. add u in the adjacency list of vertex v of the new graph. Thus traversing lists of all vertices of main graph we can get the transpose graph. Thus the total time complexity of the algorithm is $O(V+E)$ where V is number of vertices of graph and E is the number of edges of the graph. Note : It is simple to get the transpose of a graph which is stored in adjacency matrix format, you just need to get the transpose of that matrix.

Implementation:

C++

Java

Python3

C#

JavaScript

```
// CPP program to find transpose of a graph.
#include <bits/stdc++.h>
using namespace std;

// function to add an edge from vertex source to vertex dest
void addEdge(vector<int> adj[], int src, int dest)
{
    adj[src].push_back(dest);
}

// function to print adjacency list of a graph
void displayGraph(vector<int> adj[], int v)
{
    for (int i = 0; i < v; i++) {
        cout << i << "--> ";
        for (int j = 0; j < adj[i].size(); j++)
            cout << adj[i][j] << " ";
        cout << "\n";
    }
}

// function to get Transpose of a graph taking adjacency
// list of given graph and that of Transpose graph
void transposeGraph(vector<int> adj[],
                    vector<int> transpose[], int v)
{
    // traverse the adjacency list of given graph and
    // for each edge (u, v) add an edge (v, u) in the
    // transpose graph's adjacency list
    for (int i = 0; i < v; i++)
        for (int j = 0; j < adj[i].size(); j++)
            addEdge(transpose, adj[i][j], i);
}

int main()
{
    int v = 5;
    vector<int> adj[v];
```

```

addEdge(adj, 0, 1);
addEdge(adj, 0, 4);
addEdge(adj, 0, 3);
addEdge(adj, 2, 0);
addEdge(adj, 3, 2);
addEdge(adj, 4, 1);
addEdge(adj, 4, 3);

// Finding transpose of graph represented
// by adjacency list adj[]
vector<int> transpose[v];
transposeGraph(adj, transpose, v);

// displaying adjacency list of transpose
// graph i.e. b
displayGraph(transpose, v);

return 0;
}

```

Output

```

0--> 2
1--> 0  4
2--> 3
3--> 0  4
4--> 0

```

Time Complexity:

The time complexity of the addEdge function is $O(1)$, as it simply appends an element to the vector.

The time complexity of the displayGraph function is $O(V + E)$, where V is the number of vertices and E is the number of edges, as it needs to traverse the adjacency list of each vertex and print out the adjacent vertices.

The time complexity of the transposeGraph function is also $O(V + E)$, where V is the number of vertices and E is the number of edges, as it needs to traverse the

adjacency list of each vertex and add the corresponding edges to the transpose graph's adjacency list.

Therefore, the overall time complexity of the program is $O(V + E)$.

Space complexity:

In terms of space complexity, the program uses two arrays of vectors to represent the original graph and its transpose, each of which has a size of V (the number of vertices). Additionally, the program uses a constant amount of space to store integer variables and temporary data structures. Therefore, the space complexity of the program is $O(V)$.

Note that the space complexity of the program could be larger if the input graph has a large number of edges, as this would require more memory to store the adjacency lists.

 Comment

More info 

Advertise with us

Next Article 

Difference Between Graph and Tree

Similar Reads

Graph Algorithms

Graph algorithms are methods used to manipulate and analyze graphs, solving various range of problems like finding the shortest path, cycles detection. If you are looking for difficulty-wise list of problems,...

 3 min read

Introduction to Graph Data Structure

Graph Data Structure is a non-linear data structure consisting of vertices and edges. It is useful in fields such as social network analysis, recommendation systems, and computer networks. In the field of sports...

 15+ min read

Graph and its representations

A Graph is a non-linear data structure consisting of vertices and edges. The vertices are sometimes also referred to as nodes and the edges are lines or arcs that connect any two nodes in the graph. More...

 12 min read

Types of Graphs with Examples

A graph is a mathematical structure that represents relationships between objects by connecting a set of points. It is used to establish a pairwise relationship between elements in a given set. graphs are widely...

🕒 9 min read

Basic Properties of a Graph

A Graph is a non-linear data structure consisting of nodes and edges. The nodes are sometimes also referred to as vertices and the edges are lines or arcs that connect any two nodes in the graph. The basi...

🕒 4 min read

Applications, Advantages and Disadvantages of Graph

Graph is a non-linear data structure that contains nodes (vertices) and edges. A graph is a collection of set of vertices and edges (formed by connecting two vertices). A graph is defined as $G = \{V, E\}$ where V i...

🕒 7 min read

Transpose graph

Transpose of a directed graph G is another directed graph on the same set of vertices with all of the edges reversed compared to the orientation of the corresponding edges in G . That is, if G contains an...

🕒 9 min read

Difference Between Graph and Tree

Graphs and trees are two fundamental data structures used in computer science to represent relationships between objects. While they share some similarities, they also have distinct differences th...

🕒 2 min read

BFS and DFS on Graph



Cycle in a Graph

